

ID2221 Data-Intensive Computing

Lab assignment Week 2

Alexía Margrét Axelsdóttir
Gudný Eva Eiríksdóttir
Kinga Anna Koltai

September 20, 2026

Introduction

This report outlines our design of querying and optimizing the urban data platform. It is an extension of last week's integrated data platform. In this part, we developed reusable analytical queries and optimized them for performance, as well as designed and implemented reusable analytical data products. This marks the second of four parts in our continuous lab assignment; querying and optimizing the data platform.

Task 1. and 2. Design and implement analytical queries

Monthly taxi demand for each taxi zone.

We interpret taxi demand as the number of taxi trips, regardless of passenger_count. To get the demand for each taxi zone, we can group the integrated_taxi_trips by the zone pickup_zone, and select the count of entries belonging to each zone, which we name **num_trips**. Then to get the entries by month, we group the entries by the pickup month as well. For that, we use pickup_date and use the 'YYYY-MM' format, so that we can group by the month of each year, and we name that **pickup_month**. The result is a table with the columns **pickup_month**, **pickup_zone** and **num_trips** and we order the result by descending **num_trips** to display the highest demand first.

Average trip distance under different weather conditions.

Our integrated dataset includes the field pickup_weather_condition_code which is the Meteostat weather condition code that indicates the weather conditions. We use the official Meteostat documentation of the code to weather condition mapping and write the table into raw_coco_labels.csv. The codes are mapped

to 27 possible conditions, with the different scales of the weather as well (light, normal or heavy). We bucket the conditions into a broader category, so all different types of rain get stored as rain and all types of snowfall get stored as snow. We store the weather condition to broader category in `coco_buckets.csv`. We can then join our integrated dataset and raw coco labels using the condition code, and then join the buckets using the weather condition. Then we can group the entries by the broader weather conditions label, which we name **weather**. We display the average trip distance by using the `avg()` sql function on the `trip_distance` values and round that up to nearest 2 decimals, and name that field **miles**. We optionally display the count of entries belonging in each category and name that **trips**. The result is a table with the columns **weather**, **miles**, and **trips**, and we order by descending miles to get the most popular weather condition first.

Relationship between air quality and taxi demand.

We only have PM2.5 data for air quality, which spans a very wide range, making it difficult to categorize across trips. Therefore, we used the EPA's [AQI break-points table](#), which maps the PM2.5 into categories. This allows us to group the trips by air quality categories. However, the amount of trips does not reflect the demand, since some air quality categories may occur more often than others. Therefore, we need to find the number of hours each air quality category spans, which then allow us to calculate the average demand per air quality category.

Taxi zones with the largest variation in demand under different weather conditions.

We start by aggregating the trips by taxi zones and weather conditions. We then calculate the average trip count across its different weather conditions for each taxi zone. To find out how distributed the trips are in each taxi zone, we calculate the standard deviation. Finally, to be able to make this comparable between the zones, we calculate the coefficient of variation. This gives us the percentage of trip variation under different weather conditions, the higher the percentage, the greater the variation.

Peak travel hours for each day of the week.

To get the hourly trips for each day of the week, we can group the integrated dataset by the hour and day of the week and showing the count of entries for each group. To get the day of the week from `pickup_time_local`, we can use `date_format` with the format 'EEEE' to display 'Monday' to 'Sunday', and name that field **dow**. We use `hour()` to extract the hour from the pickup time and name that **hour**. The count we name **trips**. We then have the information we need, but we need to get the top hours of each day in terms of trips. For that we must use partitioning and rank. We get the rank by partitioning by `dow` and ordering by descending trips, showing how each hour ranks per day in terms

of trips, and name that **rank**. Then we can filter the result to show only the entries in the top 3 rank, so each day of the week has their top 3 busiest hours.

Monthly trends in taxi demand.

While monthly taxi demand gives an indication of the number of trips that take place, several additional metrics can be used to examine how these trips vary between months. Some interesting metrics include the average distance of trips, average passenger count per trip, the proportion of trips to or from the airport, and total revenue. To calculate these metrics, we group the integrated taxi trips by pickup month using the 'YYYY-MM' format. We then count the trips to obtain the total monthly demand, calculate the averages for trip distance, passenger count, and trip duration, and sum the total amount to obtain the total monthly revenue. The proportion of airport trips is calculated based on trips with an airport fee. We restrict the analysis to January through March 2024 to filter out occasional erroneous or out-of-scope data and order the results chronologically by pickup month.

Task 3. Optimize Query Performance

Caching

Caching can improve query performance when the same table or intermediate result is accessed several times. Our six queries don't naturally reuse an intermediate result, so instead, the experiment focuses on running multiple queries in a row that access the same integrated table. First, the six queries are executed in sequence without caching and the total execution time is recorded. We then cache the integrated table and materialize the cache before executing the same six queries again. By caching the integrated taxi trips table, Spark can reuse the cached data instead of repeatedly reading the underlying Delta data. The total execution times from the two experiments are then compared to evaluate the effects of caching.

Partition pruning

Partition pruning can reduce query execution time by skipping partitions that cannot contain relevant records. This is particularly useful for queries that filter on the partition column. The only query out of the six that filtered on the partition column was query 6, which used it to drop out-of-scope values. For the purpose of investigating partition pruning and its effect on execution time, another version of query 6 was developed which still drops out-of-scope entries, but does so using the `pickup_time_local` instead of our partition column `pickup_date`. This way, the modified query does not use partition pruning. The two versions were run one after the other and execution times were compared. Note, however, that the number of out-of-scope records is minimal compared to the size of the taxi trips dataset.

Broadcast joins

Broadcast joins can provide performance improvements when a larger table is joined with one or more much smaller tables. It works by broadcasting a copy of the small table(s) to each worker, which allows them to join its portion of the large table with the small table locally, reducing the amount of data that needs to be transferred. Query 1 offers a suitable experiment since it uses the taxi zone lookup table, which is significantly smaller than the rest, however, since we store the integrated table as well, it is not the best choice to demonstrate broadcast joins. Instead, query 2 was chosen, which uses the `coco_bucket.csv` to map weather condition scores to their names in merely 28 rows. For the baseline, automatic broadcasting was disabled and the original query was run. Then, another query version, containing a broadcast hint for Spark to utilize broadcast joins, is executed and their execution times are compared and the result verified.

Adaptive Query Execution (AQE)

AQE is a feature in Spark that allows Spark to adjust its query execution plan while the query is running, based on information it learns about the data during execution. This can help optimize operations such as shuffle partitioning, join strategies, and data skew. For this experiment, we use query 1 about monthly taxi demand, which groups the taxi trips by pickup month and pickup zone. This operation requires data to be shuffled according to the grouping columns, making it suitable for investigating the effect of AQE.

Task 4. Build Reusable Data Products

Taxi Zone Statistics

The taxi zone statistics table is most useful for taxi companies, since it shows where the highest demand is, by hour and day of the week, and shows both the total number of trips and the daily average per zone. Additional information, such as average trip duration, fare amount, and trip distance, can help determine how taxis should be distributed across zones to meet the demand. This table should be materialized rather than computed on demand, as the trip data is very large and the underlying data doesn't change frequently, so recomputing it on every query would be an unnecessary use of resources.

Air Quality Impact Summary

The air quality impact summary table could be very useful for an environmental health researchers, since it shows the correlation between air quality and the number of taxi trips. It displays the average trips per hour and the average trip distance, which can show whether people take taxi trips shorter distance when

the air quality is bad. This table should be materialized because it combines two separate datasets through a join that is expensive to repeat on every query.

Borough Mobility Summary

The borough mobility summary table could be useful for transit planners in New York. It shows the most common pickup zone, drop-off zone, and drop-off borough, which can indicate areas that may lack good public transportation. The average passenger count per trip can show how efficiently taxis are being used or if it is mainly used for solo trips. The table also identifies the most common hour and weekday for trips, which can help planners determine when more frequent trips may be needed. This table should be materialized rather than computed on demand so it can serve as a stable baseline for transit planning. When changes are made, such as adding a new bus route in an area lacking public transportation, planners can compare future data against this table to see if there were any improvements.

Weekday Mobility Summary

The weekday mobility summary table could be useful for the average person commuting within New York city. It can give them an idea of when and where it might be hard to find a taxi on a given day of the week. Additionally, the average trip duration per day can help indicate whether traffic is worse on certain days than others. The average fare amount can also help people roughly see whether a trip will fit within their price range. This table should be materialized because calculating averages by weekday requires scanning the full trip history, which is expensive to repeat on every demand. Materializing it also allows people to look up trip data quickly, rather than having to recompute it each time.

Conclusion

Through the design and implementation of our querying and optimizing data platform, we have been introduced to the main concepts of query performance and analytical data product design. This showed us how much a query's performance can change depending on how it is optimized, especially when working with very large datasets. With optimized query performance and reusable data products, we can keep the operational costs low and make the platform easier to maintain and expand when new analytical needs come up.